

F215 Digital Design — Tutorial 3

Subtractors, adder/subtractor design, scaling of adder circuits, and reading Verilog structurally

Question 1 — Building a Faster Subtractor

Two 1-bit values X and Y are to be subtracted ($X - Y$).

- (a) Derive the half-subtractor truth table (D = difference, B = borrow) from first principles and find its minimal SOP via K-map — don't just recall it.
- (b) Prove algebraically (not just by citing the diagram) that a full subtractor is exactly two half-subtractors plus an OR gate: starting from $D_1 = X \oplus Y$, $B_1 = X'Y$, feed D_1 and B_{in} into a second half-subtractor to get D_2 , B_2 , and show D_2 and $(B_1 + B_2)$ match the standalone full-subtractor equations.
- (c) Just as carry-lookahead replaces rippling with a direct two-level circuit using “generate” G_i and “propagate” P_i signals, define analogous borrow-generate G_i and borrow-propagate P_i signals (as Boolean functions of X_i , Y_i) so that $B_{i+1} = G_i + P_i B_i$ matches your part (a) equation.
- (d) Using your G_i , P_i , write the direct (non-recursive) expression for B_4 in terms of $G_0..G_3$, $P_0..P_3$, and B_{in} — mirroring the C_4 expression from lecture.

Question 2 — A Life-Counter Bug

A hardware engineering team at a game studio is investigating a bug report about the circuit that updates a player's session life tally — net lives gained minus lives lost so far this session (which can legitimately be negative if a player has net-lost lives). The tally is computed by an S-controlled adder/subtractor circuit from lecture ($S=0$ adds, $S=1$ subtracts), but the bit-width used was never documented in the legacy code. Quality assurance has logged the following:

Tally before	Event	Tally shown after
1	Gained 1 life	2
2	Gained 2 lives	-4
3	Gained 1 life	-4
2	Lost 1 life	1
-3	Lost 3 lives	2
-4	Lost 3 lives	1

- (a) Determine the bit-width being used here — the data only makes sense for one specific value. Then explain, in general terms, why certain rows give a wrong result and others don't (no need to trace bit-by-bit; describe the condition under which this circuit fails).
 - (b) Using only signals already present in the circuit (sum bits and internal carries — no extra adders), design the minimal additional logic for a flag that correctly indicates whenever this failure occurs. Verify against the table.
 - (c) Does your flag need separate handling for the subtract case ($S=1$, losing lives), or does it already work there unmodified? Justify using how the circuit internally handles subtraction, and check against the relevant rows in the table.
-

Question 3 — Reading Circuits, Not Writing Them

(a) Determine what this module computes, and simplify:

```
module mystery(input p, q, r, s, output f);
  wire w1, w2, w3;
  and (w3, w1, r);
  or  (f, w3, w2);
  and (w1, p, q);
  or  (w2, q, s);
endmodule
```

(b) A colleague builds a full subtractor from two half_subtractor instances (ports .x, .y, .d, .b) like this:

```
module full_subtractor(input x, y, bin, output d, bout);
  wire d1, b1, b2;
  half_subtractor HS1(.x(x), .y(y), .d(d1), .b(b1));
  half_subtractor HS2(.x(bin), .y(d1), .d(d), .b(b2));
  or (bout, b1, b2);
endmodule
```

Draw the circuit this code describes, labeling every wire. Then determine: is d correct for every input combination? Is bout? If not, find a specific (x, y, bin) where it fails, and identify exactly which port connection is wrong.

(c) A testbench for full_subtractor is written as:

```
module tb_full_subtractor;
  wire t_x, t_y, t_bin;
  reg  t_d, t_bout;

  full_subtractor DUT(.x(t_x), .y(t_y), .bin(t_bin), .d(t_d), .bout(t_bout));

  initial begin
    t_x = 0; t_y = 0; t_bin = 0;
    #5 t_x = 0; t_y = 1; t_bin = 0;
  end
endmodule
```

Will this simulate correctly? If not, identify every incorrect declaration and explain — from what reg and wire actually represent, not “because that’s the rule” — why each one is wrong.

Question 4 — How Do Adders Scale?

(a) For an n-bit ripple-carry adder (n full adders, each a fixed small number of gates; worst-case delay $2n+1$ gate-levels, from lecture), express total gate count and delay as functions of n. State how each grows as n increases.

This kind of growth-rate description is usually written using Big-O notation: $O(n)$ means “grows roughly proportionally to n,” $O(n^2)$ means “roughly proportional to n^2 ,” $O(\log n)$ means “grows much more slowly —

roughly the number of times you can repeatedly divide n by some fixed factor before reaching 1," and $O(1)$ means "stays roughly constant regardless of n ."

(b) Now take the same generate/propagate circuit from lecture and imagine building it for a general n -bit adder, the same way, just bigger. How do its total gate count and its delay scale with n ? (Trying it out for a few small values of n first may help before generalizing.)

(c) In practice, adders are built by taking 4-bit CLA units (as in lecture) and combining them using the carry-out of one feeding the carry-in of the next — the same way full adders were chained into a ripple-carry adder. For a 64-bit adder built this way (16 such 4-bit blocks), express the total worst-case delay in terms of the number of blocks and whatever per-block delay you found relevant in (b). How does this scale with n ? Compare it to both (a) and (b) — what has this approach actually improved, and what hasn't it changed?

(d) Here's a way to actually change the growth rate. Apply the same generate/propagate trick to the blocks themselves: define a block-generate signal G_{blk} (this block produces a carry regardless of its incoming carry) and a block-propagate signal P_{blk} (an incoming carry sails straight through the whole block), each as a Boolean function of that block's own 4 bit-level G_i, P_i . A second-level lookahead unit — structurally identical to the one from (b), just one level up — can then compute every block's incoming carry directly, instead of rippling block to block. Applied recursively: argue that overall delay becomes $O(\log n)$ while total gate count stays $O(n)$ (a structural argument is enough — no rigorous proof needed). For $n=64$ with 4-bit blocks: how many levels of hierarchy are needed? Roughly how many gate-delays does this give you, compared to the 129 from lecture?